PACMAN Quality Assurance and Quality Control

Firmware validation; hardware checkout: tools, coverage, and shifter's guide

Draft maintained by the PACMAN agents (augie); reference for the TRM

2026-10-18

1 Purpose and scope

- Define a repeatable checkout of every new PACMAN card before it goes into a production system or onto an ASIC test stand.
- Two utilities, both run on the PACMAN's own Linux system:
  - `pacman_lnxcheck`: the standard embedded-Linux checkout of the Trenz module (CPU, memory, on-chip supplies, Ethernet, flash and SD storage).
  - `pacman_hwcheck`: the PACMAN-specific checkout (power, tile I/O, timing inputs, front panel), partly guided by an operator.
- Firmware validation (Section 7), separately: once per firmware release, against stated requirements, in simulation and on a bench Trenz module. It qualifies the firmware package, not a card.
- Scope: the features needed for normal operation. Features that normal operation does not use are listed (Section 6) but not tested.
- Method: wherever possible a feature is tested by loopback, so that the feature and its feedback path are verified together, and a failure points at hardware, firmware or software.

2 Production setting

- PACMAN 1v5 with a Trenz TE0720 module (Zynq-7000). The card boots from its microSD card (boot image and Linux root file system) and is reached over Ethernet only (`ssh`). Production cards have no USB/JTAG header installed.
- **ASIC protection jumper (J5).** Installed whenever the card is in the production system or connected to ASICs. The checkout runs with J5 *removed*; with J5 installed, `pacman_hwcheck` refuses everything except its read-only report.
- The PACMAN data and command servers are stopped during the checkout (they share the data path with the loopback test) and restarted afterwards.

3 Test setup

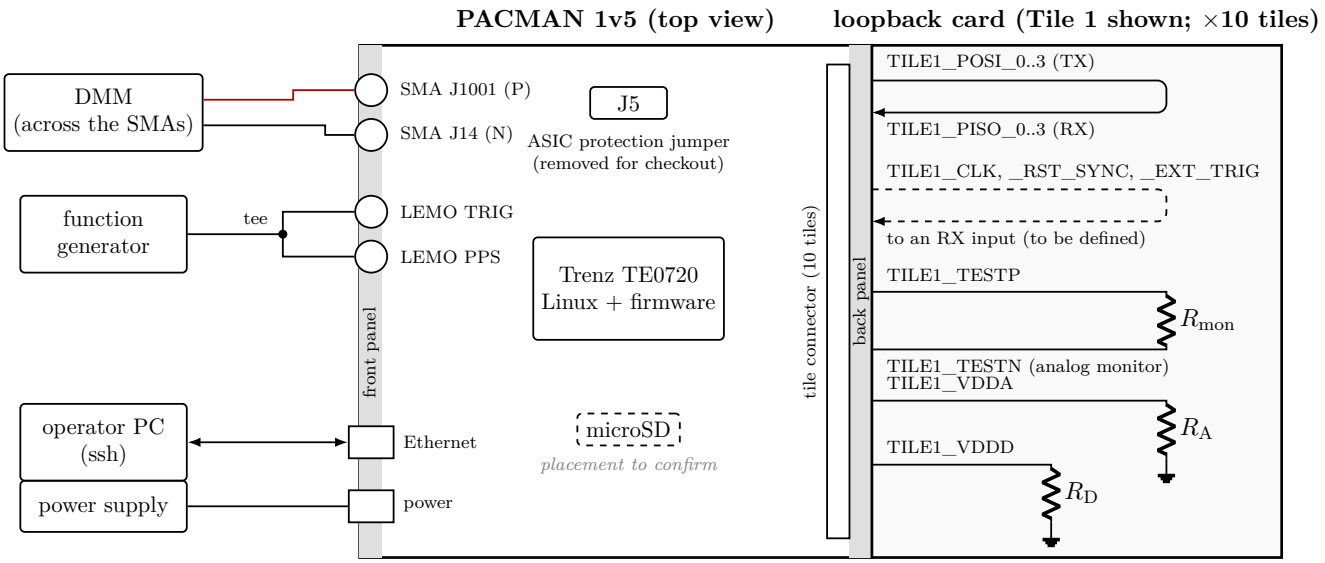


Figure 1: Checkout setup (physical view; connector placement to be confirmed). All loopbacks are on the loopback card; Tile 1's connections are shown with their signal names, and the card repeats them for all ten tiles. Dashed: routing still to be defined.

Item	Role
Loopback card	On the back-panel tile connector, for all ten tiles: each tile's data outputs (POSI) loop back to its data inputs (PISO); its clock, sync and trigger outputs loop back to an RX input; a resistor closes its analog monitor pair (TESTP/TESTN); load resistors on VDDA and VDDD let the tile supplies be swept under load.
DMM	Front-panel SMA readings, entered by the operator when prompted.
Function generator	One output, split with a tee: the same square wave on the TRIG and PPS LEMO inputs.
Operator PC	`ssh` session to the PACMAN; keeps the result files.

- Two versions of the loopback card are needed for different tests, so the checkout is split into one run per card, with the PACMAN powered down while the cards are swapped. Which tests go on which card: [TBD: lab day].

4 The utilities

4.1 Common conventions

- One line per result: `<VERDICT> <name>-- <value>`, then a final `VERDICT` line.
- Result verdicts: `PASS`, `FAIL`, `SKIP` (not done, with the reason), `INFO` (recorded, not judged). `pacman_hwcheck` also prints `DATA` lines (sweeps).
- Run verdict and exit code:

Run verdict	Meaning	Exit code
`PASS`	every check passed, nothing skipped	0
`FAIL`	at least one check failed	1
`INCOMPLETE`	no failure, but something was skipped	2
(refused / usage)	bad options, or J5 installed	3

- Only a `PASS` run qualifies a card.

4.2 `pacman_lnxcheck`: embedded Linux

Section	Checks
System	CPU count, memory size, kernel version.
On-chip monitors	Die temperature; six Zynq supplies against nominal.
Ethernet	Link speed and duplex (1000/full), MAC address, IP address; ping with small and full-size packets, no loss; TFTP transfer of a file twice, compared; interface error counters unchanged.
QSPI flash	Each partition read three times; the last two reads must agree. A difference on the first read after boot is reported (INFO), not failed.
eMMC	First 64 MB read twice, compared; read rate reported.
microSD	Card present, partitions, boot image checksum. Optional `--sd_write`: a 16 MB test file written, read back and removed.

Read-only unless `--sd_write` is given. Options: `--server` (TFTP server; default the default gateway), `--tftp_file`, `--no_tftp`, `--sd_write`.

4.3 `pacman_hwcheck`: PACMAN hardware

Run sets execute in this fixed order; `--full_checkout` runs all of them and stops at the first failing section (`--continue` keeps going).

Run set	Checks	Operator
`--read_only`	Firmware and hardware versions, build information, J5 state, board and tile monitors, data path status. Allowed with J5 installed.	–
`--blink`	Trenz and PACMAN LEDs blink in turn.	watches, confirms
`--power`	Board rails against nominal; tile power off, then on. Each tile in turn: VDDA and VDDD swept over the DAC range into the card's loads (floor, gain, swing, monotonic, load), other tiles stay off. Board-rail test points surveyed with the DMM.	DMM readings
`--loopback`	Each tile in turn, only that tile enabled: random data on its four UARTs through the loopback card, words received, bit errors, receiver error counters; no stray data on other UARTs.	–
`--timing_inputs`	Rates on the TRIG and PPS inputs, counted by the firmware over 1 s.	–
`--analog_io`	Front-panel analog MUX stepped through tiles 1–10, test DAC, off; a DMM reading at the SMA jacks for each.	DMM readings

- Operator steps start with a prompt saying what comes next (`Ready?  [y = go, s = skip]`). At any prompt, `s` skips the step; it is recorded as SKIP, never as a value.

- Modifiers: `--tile_mask` (tiles present on the fixture), `--skip_interactive` (no operator steps), `--continue`; development: `--internal_loopback`, `--no_i2c`, `--sim_i2c`, `--ber_bits`, `--analog_io_mode`.

5 Coverage and acceptance limits

Feature	Acceptance	Status
Board rails 3V6, 3V3, 3V0	monitor within 5% of nominal; DMM within 5% of monitor; 3V0 $\leq$ 50 mV when off	limits set; DMM test points [TBD:]
Tile supplies VDDA, VDDD	floor and gain (157 mV per 0x1000 DAC codes, $\pm$5%), swing $\geq$ 1 V, monotonic, current under load, other tiles $\leq$ 50 mV	provisional (one board)
Tile UARTs (40)	every word received, zero bit errors, no stray words	set
TRIG, PPS inputs	rate matches the generator	rates recorded; limits [TBD:]
Front-panel analog MUX	expected SMA readings	readings recorded; values [TBD:]
LEDs	operator confirms	set
Linux, Ethernet, storage	as in the `pacman_lnxcheck` table	set

6 Status and untested features

Needed for normal operation, not yet tested	Plan
Tile CLK, TRIG, SYNC outputs	Loopback to an RX input on a separate connector (fixture [TBD:]).
Analog monitor	Known resistor across the input, measured through the front panel.
TRIG / PPS acceptance	Function generator at the lab; then rate limits.
Full ten-tile run	Loopback card covering all ten tiles.
microSD checks	First run on an SD-booted card.

Not needed for normal operation (not tested): onboard ADC, RTD temperature probe, QSPI flash contents, SFP/CDR timing path.

7 Firmware validation

- Once per firmware release (official or candidate), before it is installed on production cards.
- Two stages, against one list of requirements:
  - **Simulation** (`make check`, GHDL): every requirement.
  - **Hardware** (`pacman_fwcheck`, on a bench Trenz module running the release): the requirements that can be checked on the board with no tiles and no external signals.
- Both are written by the PACMAN agents and kept in `agents/` directories, pending human review: `pacman-firmware/ghdl/agents` (requirements, checks) and `pacman-software/agents/fwcheck`.

7.1 Requirements

- Source: `pacman-firmware/ghdl/agents/requirements.txt`; listed in Appendix A.
- Three levels:
  - Level 1 (`RX-1`): what a unit promises, for a human to read.
  - Level 2 (`RX-1.1`): a testable statement, backed by checks.
  - Level 3: the individual checks of the GHDL check benches (`check/*_check.vhd`), each counted under the level-2 requirement in force, set in the bench with `req("RX-1.1")`.
- Each level-2 requirement is marked `sim` (simulation only: internal timing, reset, pins, external inputs) or `sim+hw` (also checked on the board by `pacman_fwcheck`).

Unit		Level 1	Level 2 `sim`	Level 2 `sim+hw`
`REG`	Register access (all units)	4	4	6
`GLB`	Global unit	2	3	2
`TX`	TX unit	3	3	6
`RX`	RX unit	6	7	12
`ATC`	Timing unit	5	6	4
	Total	20	23	30

7.2 Simulation: `make check`

- Run in `pacman-firmware/ghdl/agents`: `make check` (GHDL; about 20 s). Five check benches (Global, TX, RX, Timing, and the register bus across all units), 3300 checks.
- Output: one line per bench, then one line per requirement, under its level-1 heading:

```
PASS rx_unit_check (671 checks)
...
  REG-2    Registers hold what is written, independently.
    PASS REG-2.1    161  sim+hw  Every read/write register reads back every value
                                 written to it, all bits.
...
---- 53 requirements, 53 with checks, 3300 checks in all, 0 to look at
```

- Per requirement: `PASS`, `FAIL` (a check failed), or `NONE` (no check backs it). Also flagged: requirement IDs not in `requirements.txt`, and checks made before any `req()`.
- Pass: every bench `PASS`, every requirement `PASS`, `0 to look at` (exit code 0).

7.3 Hardware: `pacman_fwcheck`

- Checks the 30 `sim+hw` requirements in the installed firmware. One source, two builds: Linux (a static binary, independent of the root file system) and bare metal (JTAG).
- Method: the firmware's own loopback. DMA packets go out through the TX unit, back in through RX input 2 (internal loopback) and out through DMA; masks, all 40 channels, every bit, 1000 random packets, frame length, headers, packet size and timeout. Also: register read-back and crosstalk, broadcast writes, timeouts, the timestamp rate, the test pattern, heartbeat and marker words, SYNC, and the timing unit's counters (pokes routed to the markers only).

- Safety: only the read-only checks run with J5 installed, with tile power or any tile enabled, with `--read_only`, or (Linux) while a PACMAN server runs. Every register written is saved and restored, also on Ctrl-C or kill. Not restored: the counters, the timestamp (SYNC restarts it) and the DMA rings (the servers set them up when started). Nothing leaves the PL.
- Output: failed checks as they happen (all checks with `-v`), then one line per requirement (`PASS`, `FAIL`, `NONE`: not checked) and a `VERDICT` line; verdicts and exit codes as the other utilities (Section 4.1). A read-only run is `INCOMPLETE`.

```
PASS REG-2.1    54  every read/write register reads back every value written
...
VERDICT PASS  (30 requirements:  0 failed, 0 not checked;  0 failed checks)
```

- Run on Linux (about 0.2 s):
  `/etc/init.d/pacman_server stop; ./pacman_fwcheck; /etc/init.d/pacman_server start`
- Build and bare-metal run: `pacman-software/agents/fwcheck/README.txt`.

7.4 Release procedure

A firmware release is validated when, for that release:

1. `make check` passes (all 53 requirements);
2. `pacman_fwcheck` gives `VERDICT PASS` on a bench Trenz module running the release's bitstream, under Linux and on bare metal.

The three outputs are kept with the release. First result (3.6.0 candidate, git 13dc480, 2026-10-17): all pass.

7.5 Coverage

- The 23 `sim` requirements are checked in simulation only: they concern internal timing (register bus timing, UART phase and delay), reset, the board pins (tile enables, LEDs, G and H outputs), the LEMO inputs and the receiver's tolerance of real lines (glitches, frame errors, stalls). On each card, `pacman_hwcheck` covers the tile enables, LEDs and LEMO inputs; the G and H tile outputs are not yet tested (Section 6).
- The simulation is of the firmware sources, the hardware check of the built bitstream: together they cover the build as well as the design.

8 Shifter's guide: hardware checkout

For an operator who runs the checkout the standard way. Items marked [TBD: ...] are still to be filled in.

You need

- The PACMAN card with its microSD card (production image), and its serial number [TBD: where printed].
- Loopback / load card, DMM, function generator with two outputs, Ethernet cable, PACMAN power supply.
- A PC on the same network, with `ssh`.

Prepare

1. Power off. Remove J5 (ASIC protection jumper). Make sure no ASICs are connected.
2. Install the loopback / load card on the tile connectors.
3. Connect the function generator to the TRIG and PPS LEMO inputs: square wave, [TBD: amplitude], [TBD: frequency].
4. Connect Ethernet; power on. Wait about [TBD: N] s for Linux to boot.
5. Log in: `ssh [TBD: user]@[TBD: PACMAN address]`.
6. Stop the PACMAN servers: `/etc/init.d/pacman_server stop`

Run

1. Linux checkout (a minute or less):
   `pacman_lnxcheck --server [TBD: TFTP server] | tee lnxcheck-<serial>.txt`
2. Hardware checkout (guided; about [TBD: N] minutes):
   `pacman_hwcheck --full_checkout | tee hwcheck-<serial>.txt`
3. Answer the prompts:
   - `Next: ... Ready?`: get ready for what it describes (watch the LEDs, put the DMM probes on the named points), then answer `y`.
   - Yes/no questions: answer what you saw.
   - Readings: type the DMM value in mV.
   - Can't do a step? Answer `s`. It is recorded as skipped; never type a made-up value.

Read the result

- The last line of each run is the verdict.
- `PASS` from both: the card passes.
- `FAIL`: the lines marked `FAIL` say what failed. The hardware checkout stops at the first failing section. Do not repeat blindly: record, and report to [TBD: contact].
- `INCOMPLETE`: something was skipped (lines marked `SKIP` give the reason). Fix the setup and run again.
- `REFUSED` with J5 mentioned: J5 is installed; power off, remove it, start again.

Finish

1. Copy both result files to [TBD: results location], named by serial number.
2. Restart the servers (`/etc/init.d/pacman_server start`) or reboot.
3. Power off; remove the fixtures. Install J5 before the card goes to a production system or ASICs.
4. Record the card's status in [TBD: tracking sheet].

A Firmware requirements

Generated from `pacman-firmware/ghdl/agents/requirements.txt` (`requirements_tex.py`). Level-1 requirements in italics; `sim+hw`: also checked by `pacman_fwcheck`.

ID	Where	Requirement
REG: Register access (all units)		
REG-1	*Every register is reachable at its address, and nothing else answers.*	
REG-1.1	sim+hw	Each unit acknowledges reads and writes of its own defined registers.
REG-1.2	sim+hw	Undefined addresses, other units' scopes, writes to read-only registers and reads of the broadcast channel are not acknowledged, and change nothing.
REG-1.3	sim+hw	A request nobody acknowledges is completed by the bridge after its timeout (a read returns 0xEEEEEEEE), and the bus keeps working.
REG-2	*Registers hold what is written, independently.*	
REG-2.1	sim+hw	Every read/write register reads back every value written to it, all bits.
REG-2.2	sim+hw	A write changes only the register addressed: no crosstalk between registers, channels or units.
REG-2.3	sim+hw	A broadcast write (channel 0x3B) sets the register of all 40 UARTs.
REG-3	*Register access timing.*	
REG-3.1	sim	Global and Timing answer reads and writes one clock after the request; RX and TX answer writes one clock and reads two clocks after the request.
REG-3.2	sim	Back-to-back requests, to one unit or across units and clock domains, are answered in order, each once, with no mixed data.
REG-3.3	sim	A read in the same clock as a write of the same register returns the old value (current behaviour, no stated contract).
REG-4	*Reset.*	
REG-4.1	sim	Reset restores every register to its default, in every unit.
GLB: Global unit		
GLB-1	*The board controls do exactly what the register says.*	
GLB-1.1	sim	ENABLES bit 16 drives TILE_POWER_ENABLE and bits 9-0 TILE_ENABLES; the other bits drive nothing.
GLB-1.2	sim	LEDS bits 1-0 drive the two PL LEDs; the other bits drive nothing.
GLB-1.3	sim	After reset, tile power, the tiles and the LEDs are off.
GLB-2	*The firmware identifies itself.*	
GLB-2.1	sim+hw	STATUS reads 0x0000CAFE.
GLB-2.2	sim+hw	The version registers read the firmware and hardware versions and the build information.
TX: TX unit		
TX-1	*Every word of a DMA packet is sent on the UARTs of its mask, intact, and nowhere else.*	
TX-1.1	sim+hw	The channels of the mask send their own word; the others stay idle; mask bits above 39 are ignored.
TX-1.2	sim+hw	All 40 channels can send at once, each its own word.
TX-1.3	sim+hw	Back-to-back packets are sent in order, with no word dropped or repeated; the next packet is read while the UARTs send.
TX-2	*The UART frames are as configured.*	
TX-2.1	sim+hw	A frame is a start bit, the 64 data bits least significant first, and a stop bit, one bit per BAUD period.
TX-2.2	sim	DELAY adds exactly that many idle bit periods between frames.
TX-2.3	sim	PHASE moves the bit edges within the BAUD period.
TX-3	*The TX unit can be monitored.*	
TX-3.1	sim+hw	STARTS and BEATS count each channel's frames and hand-overs (equal in normal operation); ZERO_CNTS clears them.
TX-3.2	sim+hw	LOOK shows the word last loaded for the selected channel.
TX-3.3	sim	The status registers show the channel and buffer states.

ID	Where	Requirement
RX: RX unit		
RX-1	*Every word received reaches the stream once, intact, with its header and timestamp.*	
RX-1.1	sim+hw	A word on one channel comes out as header, timestamp and data, followed by the trailer.
RX-1.2	sim+hw	Words on several channels at once come out in channel order, in one packet.
RX-1.3	sim+hw	Every data bit arrives as sent.
RX-1.4	sim+hw	The header carries the word type (WORD_TYPE_LUT, by data bits 1-0), the PACMAN id and the channel id.
RX-2	*The receiver copes with real lines.*	
RX-2.1	sim	A start bit at any phase of the BAUD period is received (MODE 1).
RX-2.2	sim	A glitch shorter than a bit is outvoted, and counted as noise.
RX-2.3	sim	A bad stop bit is a frame error: counted, no word.
RX-2.4	sim+hw	MODE 0 turns a channel off; INPUT selects the line (idle, PISO, loopback, test pattern).
RX-3	*The stream is cut into DMA packets as configured, and congestion is accounted for.*	
RX-3.1	sim	When the stream stalls, waiting words are held; a further word on a channel still holding one is lost, and counted.
RX-3.2	sim+hw	BUFFER_CONFIG's maximum closes a packet after that many words.
RX-3.3	sim+hw	BUFFER_CONFIG's timeout lets later words join the open packet, then closes it.
RX-4	*The extra channels add their words when enabled.*	
RX-4.1	sim+hw	Heartbeat: a word every HEARTBEAT_CONFIG clocks (HEADER_A, timestamp, zero data).
RX-4.2	sim	Rollover: a word when the timestamp equals ROLLOVER_CONFIG (HEADER_B).
RX-4.3	sim+hw	Markers 2 and 3: a word for each marker pulse (HEADER_C, HEADER_D).
RX-5	*The test pattern generator feeds any channel.*	
RX-5.1	sim+hw	A channel on the test pattern receives the pattern word, frame after frame; the generator's status and counters follow.
RX-6	*The RX unit can be monitored.*	
RX-6.1	sim+hw	The counters (STARTS, BEATS, UPDATES, LOST, FRAME_ERR, NOISE) count their events; ZERO_CNTS clears them.
RX-6.2	sim+hw	LOOK shows the last word received on the selected channel.
RX-6.3	sim	The status registers show the channel and buffer states.
RX-6.4	sim	FIFO_CNT shows the FIFO count, FIFO_MAX its maximum since ZERO_CNTS.
ATC: Timing unit		
ATC-1	*The clocks and the timestamp run as configured.*	
ATC-1.1	sim	BAUD and UCLK follow CONFIG_BAUD and CONFIG_UART.
ATC-1.2	sim+hw	The timestamp counts UART periods, and reaches the RX unit.
ATC-1.3	sim+hw	Marker 0 restarts the timestamp.
ATC-2	*Each stimulus goes where its destination register sends it.*	
ATC-2.1	sim	A poke reaches G and/or H on the tiles of its destination and of its own mask, for its duration.
ATC-2.2	sim+hw	A stimulus reaches the markers of its destination.
ATC-2.3	sim	LEMO A and B, and every poke, route by their own destination registers.
ATC-2.4	sim	LOGIC A and B never fire (disabled in this version).
ATC-3	*The G and H outputs are shaped as configured.*	
ATC-3.1	sim	CONFIG_G and CONFIG_H set the phase of the outputs within the UART period, and invert tiles.
ATC-4	*The LEMO inputs give one stimulus per clean edge.*	
ATC-4.1	sim	A rising edge held two clocks or more is one stimulus; glitches and bounces are not.
ATC-5	*The counters count, and are controlled by their commands.*	
ATC-5.1	sim+hw	READ latches the edges and the integral of the chosen signal; START, STOP and CLEAR act on all.